PGR210– Lec 4
Kristiania University of Applied Sciences

By Huamin Ren

Huamin.ren@kristiania.no

Outline

- 1. Challenges
- 2. Building your vocabulary with a tokenizer
  - 2.1 Dot product
  - 2.2 Measuring bag-of-words overlap
  - 2.3 A token improvement
  - 2.4 Extending your vocabulary with n-grams
  - 2. 5 Normalizing your vocabulary
- 3. Bag-of-words (BoW)

What you learn today:
Split a document, any string, into discrete tokens of meaning. Our tokens are limited to words, punctuation marks, and numbers, but the techniques we use are easily extended to any other units of meaning contained in a sequence of characters, like ASCII emoticons, Unicode emojis, mathematical symbols, and so on.

Høyskolen Kristiania

Think for a moment: what need to be done for retrieving tokens?

- Discussion

Hint:

Split into words
Separate punctuation from words, like quotes at the beginning and end of a statement
Split contractions like “we’ll”
Include all tokens in the vocabulary
Return to the regular expression toolbox to try to combine words with similar meaning - stemming
Assemble a vector representation of documents called a bag of words
Use this bag of words vector to see if it can help improve upon the greeting recognizer

Høyskolen Kristiania

Outline

- ***1. Challenges***
- 2. Building your vocabulary with a tokenizer
  - 2.1 Dot product
  - 2.2 Measuring bag-of-words overlap
  - 2.3 A token improvement
  - 2.4 Extending your vocabulary with n-grams
  - 2. 5 Normalizing your vocabulary
- 3. Sentiment

Høyskolen Kristiania

1. Challenges

- Stemming: grouping the various inflections of a word into the same “bucket” or cluster

Høyskolen Kristiania

Outline

- 1. Challenges
- ***2. Building your vocabulary with a tokenizer***
  - 2.1 Dot product
  - 2.2 Measuring bag-of-words overlap
  - 2.3 A token improvement
  - 2.4 Extending your vocabulary with n-grams
  - 2. 5 Normalizing your vocabulary
- 3. Sentiment

Høyskolen Kristiania

2. Building your vocabulary with a tokenizer

- In NLP, tokenization is a particular kind of document segmentation. Segmentation breaks up text into smaller chunks or segments, with more focused information content
- Segmentation can include breaking
  - a document into paragraphs
  - paragraphs into sentences
  - sentences into phrases
  - phrases into tokens (usually words) and punctuation

Høyskolen Kristiania

2. Building your vocabulary with a tokenizer

- Lexicon: the vocabulary - the set of all the valid tokens
- Tokenization is the ***first step*** in an NLP pipeline

Høyskolen Kristiania

2. Building your vocabulary with a tokenizer

- Create a numerical vector representation for each word

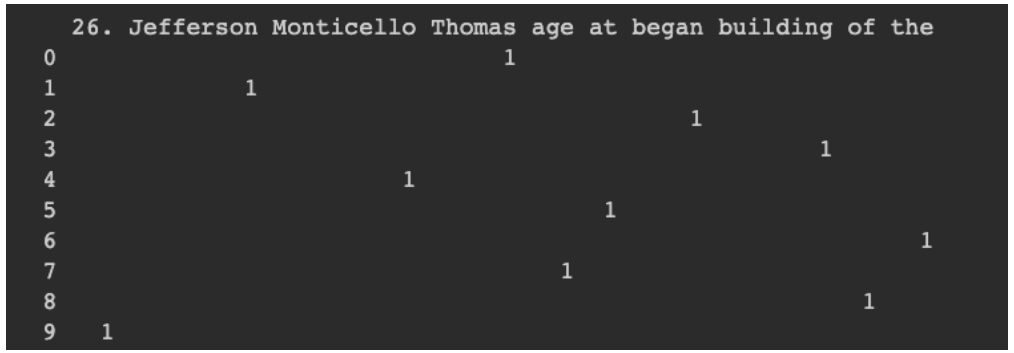
	sent0	sent1	sent2	sent3
Thomas	1.0	0.0	0.0	0.0
Jefferson	1.0	0.0	0.0	0.0
began	1.0	0.0	0.0	0.0
building	1.0	0.0	0.0	0.0
Monticello	1.0	0.0	0.0	1.0
at	1.0	0.0	0.0	0.0
the	1.0	0.0	1.0	0.0
age	1.0	0.0	0.0	0.0
of	1.0	0.0	0.0	0.0
26.	1.0	0.0	0.0	0.0

```
array([[0, 0, 0, 1, 0, 0, 0, 0, 0, 0],
       [0, 1, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 1, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 1, 0, 0],
       [0, 0, 1, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 1, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 1],
       [0, 0, 0, 0, 1, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 1, 0],
       [1, 0, 0, 0, 0, 0, 0, 0, 0, 0]])
```

Høyskolen Kristiania

2. Building your vocabulary with a tokenizer

- Create a numerical vector representation for each word



	26.	Jefferson	Monticello	Thomas	age	at	began	building	of	the
0					1					
1			1							
2							1			
3									1	
4				1						
5						1				
6										1
7					1					
8									1	
9	1									

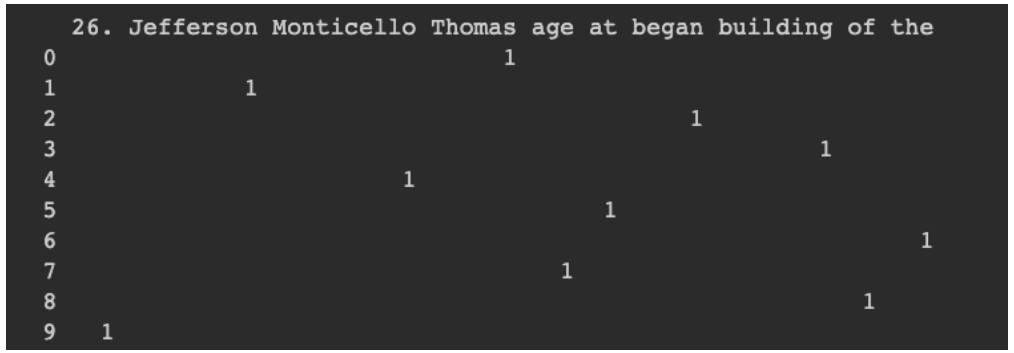
Each row of the table is a binary row vector, and you can see why it's also called a one-hot vector: *all but one of the positions (columns) in a row are 0 or blank.*

You can use the vector `[0, 0, 0, 0, 0, 0, 1, 0, 0, 0]` to represent the word "began" in your NLP pipeline.
Then what?

Høyskolen Kristiania

2. Building you vocabulary with a tokenizer

- Create a numerical vector representation for each word



	26.	Jefferson	Monticello	Thomas	age	at	began	building	of	the
0				1						
1		1								
2							1			
3									1	
4			1							
5						1				
6										1
7					1					
8									1	
9	1									

Nice features:

This vector representation of words and tabular representation of documents has no lost information.

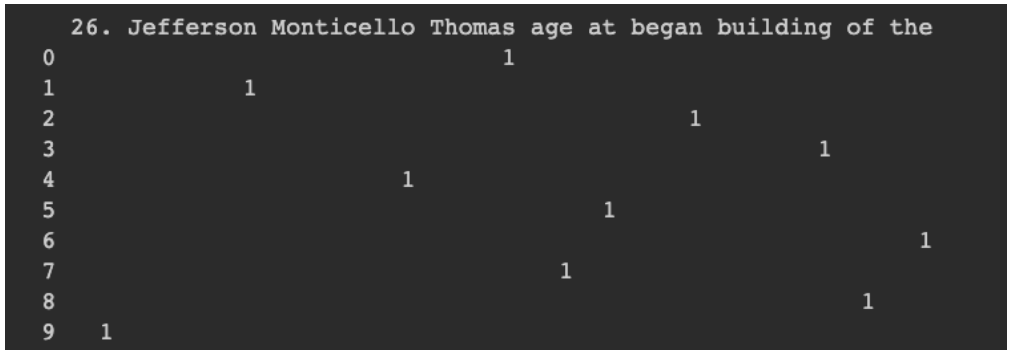
one-hot word vectors like this are typically used in neural nets, sequence-to-sequence language models, and generative language models. They're a good choice for any model or NLP pipeline that needs to retain all the meaning inherent in the original text.

Your document size (the length of the vector table) would grow to be huge ☹

Høyskolen Kristiania

2. Building your vocabulary with a tokenizer

- Create a numerical vector representation for each word



	26.	Jefferson	Monticello	Thomas	age	at	began	building	of	the
0				1						
1		1								
2							1			
3									1	
4			1							
5						1				
6										1
7					1					
8									1	
9	1									

The English language contains at least 20,000 common words, millions if you include names and other proper nouns.

Your document size (the length of the vector table) would grow to be huge ☹

Høyskolen Kristiania

Discussion: better idea?

2. Building your vocabulary with a tokenizer

- After discussions:
  - Understand One-hot

One-hot vector: each row of the table is a binary row vector - all but one of the positions (columns) in a row are 0 or blank.
Only one column, or position in the vector, is “hot” (“1”).
A one (1) means on, or hot.
A zero (0) means off, or absent.

Høyskolen Kristiania

Outline

- 1. Challenges
- 2. Building your vocabulary with a tokenizer
  - ***2.1 Dot product***
  - 2.2 Measuring bag-of-words overlap
  - 2.3 A token improvement
  - 2.4 Extending your vocabulary with n-grams
  - 2. 5 Normalizing your vocabulary
- 3. Bag-of-words (BoW)

Høyskolen Kristiania

2.1 Dot product

- Also called the inner product: the “inner” dimension of the two vectors or matrices must be the same

the number of elements in each vector

the rows of the first matrix and the columns of the second matrix

- Also called scalar product: produces a single scalar value as its output

Høyskolen Kristiania

2.1 Dot product

- The calculation: multiplying all the elements of one vector by all the elements of a second vector, and then adding up those normal multiplication products

Høyskolen Kristiania

Outline

- 1. Challenges
- 2. Building your vocabulary with a tokenizer
  - 2.1 Dot product
  - ***2.2 Measuring bag-of-words overlap***
  - 2.3 A token improvement
  - 2.4 Extending your vocabulary with n-grams
  - 2. 5 Normalizing your vocabulary
- 3. Bag-of-words (BoW)

Høyskolen Kristiania

2.2 Measuring bag-of-words overlap

- By measuring the bag of words overlap for two vectors, we can get a good estimate of how similar they are in the words they use
- Not only are dot products possible, but other vector operations are defined for these bag-of-word vectors: addition, subtraction, OR, AND, and so on

Høyskolen Kristiania

Outline

- 1. Challenges
- 2. Building your vocabulary with a tokenizer
  - 2.1 Dot product
  - 2.2 Measuring bag-of-words overlap
  - ***2.3 A token improvement***
  - 2.4 Extending your vocabulary with n-grams
  - 2. 5 Normalizing your vocabulary
- 3. Bag-of-words (BoW)

Høyskolen Kristiania

2.3 A token improvement

- In some cases, you want these punctuation marks to be treated like words, as independent tokens
- In some cases, it is the opposite

Regex?

Høyskolen Kristiania

Outline

- 1. Challenges
- 2. Building your vocabulary with a tokenizer
  - 2.1 Dot product
  - 2.2 Measuring bag-of-words overlap
  - 2.3 A token improvement
  - ***2.4 Extending your vocabulary with n-grams***
  - 2. 5 Normalizing your vocabulary
- 3. Bag-of-words (BoW)

Høyskolen Kristiania

2.4 Extending your vocabulary with n-grams

- An n-gram is a sequence containing up to n elements that have been extracted from a sequence, using a string
- 2-gram: a pair of words, like “ice cream”
- n-gram tokenizer: from nltk

I scream, you scream, we all scream for ice cream.

Høyskolen Kristiania

2.4 Extending your vocabulary with n-grams

- Wait! Stop words?!
- Stop words are common words in any language that occur with a high frequency but carry much less substantive information about the meaning of a phrase

  a, an

  the, this

How to remove them?

Mark reported to the CEO
Suzanne reported as the CEO to the board

Høyskolen Kristiania

Outline

- 1. Challenges
- 2. Building you vocabulary with a tokenizer
  - 2.1 Dot product
  - 2.2 Measuring bag-of-words overlap
  - 2.3 A token improvement
  - 2.4 Extending your vocabulary with n-grams
  - ***2. 5 Normalizing your vocabulary***
- 3. Bag-of-words (BoW)

Høyskolen Kristiania

2. 5 Normalizing your vocabulary

Case folding

Stemming

Lemmatization

Multiple ‘spelling’ of a word that differ only in their capitalization, also called ‘case normalization’.

Høyskolen Kristiania

2. 5 Normalizing your vocabulary

Case folding

Stemming

Lemmatization

Stemming: identify a common stem among various forms of a word.
Housing, houses….. -> house

Høyskolen Kristiania

2. 5 Normalizing your vocabulary

Case folding

Stemming

Lemmatization

Associate several words together with similar meanings, even though their spelling might be different.

Høyskolen Kristiania

Outline

- 1. Challenges
- 2. Building you vocabulary with a tokenizer
  - 2.1 Dot product
  - 2.2 Measuring bag-of-words overlap
  - 2.3 A token improvement
  - 2.4 Extending your vocabulary with n-grams
  - 2. 5 Normalizing your vocabulary
- ***3. Bag-of-words (BoW)***

Høyskolen Kristiania

3. Bag-of-words

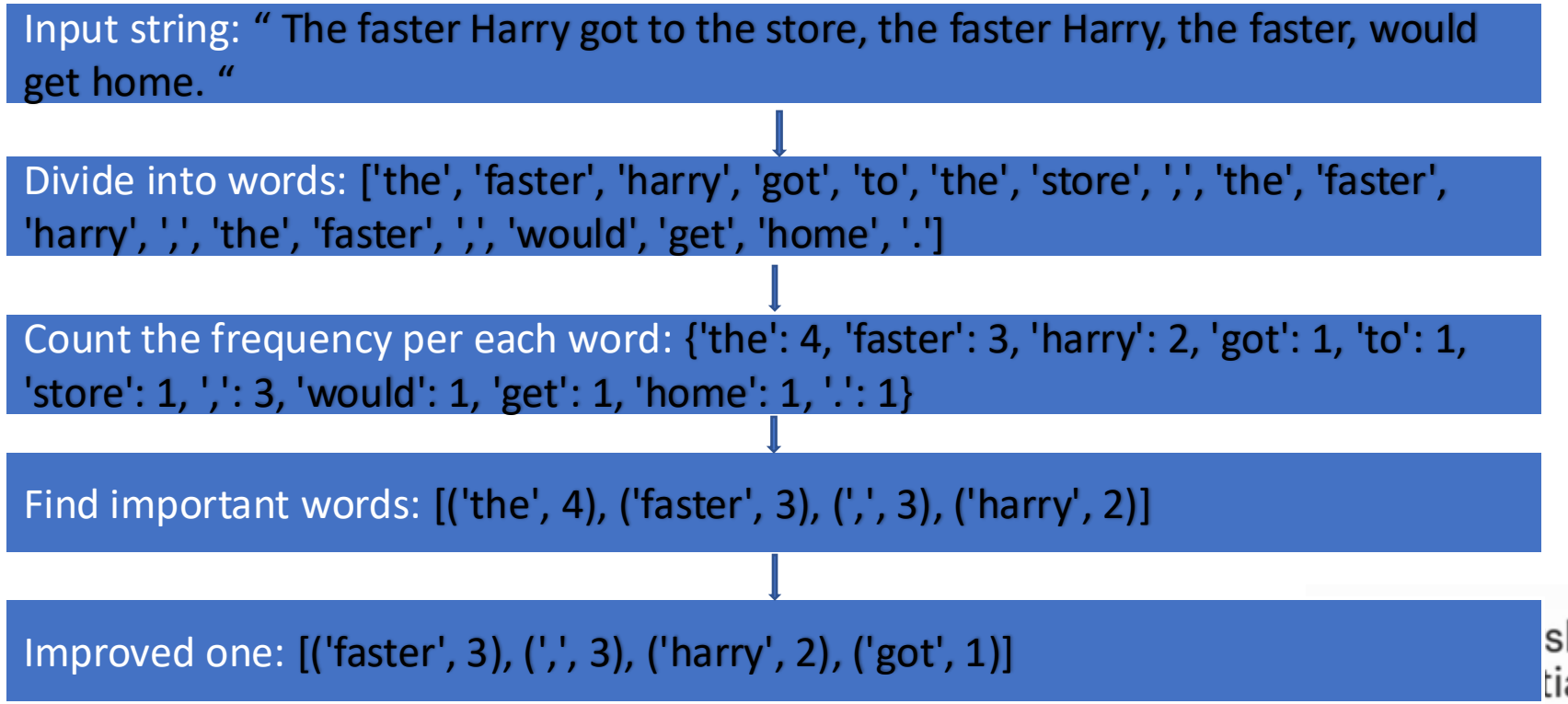
Next step: turn words into continuous numbers rather than just integers to capture the *importance*

- 3.1 Bags of words—Vectors of word counts or frequencies
- 3.2 Bags of n-grams—Counts of word pairs (bigrams), triplets (trigrams), and so on
- 3.3 TF-IDF vectors—Word scores that better represent their importance

Høyskolen Kristiania

3.1 Bags of words

- Think for a moment: how to turn ‘binary’ value to a continusous value?



3.1 Bags of words

A ***collections.Counter*** object is an unordered collection, also called a bag or multiset. Depending on your platform and Python version, you may find that a Counter is displayed in a seemingly reasonable order, like lexical order or the order that tokens appeared in your statement. But just as for a standard Python dict, you cannot rely on the order of your tokens (keys) in a Counter.

Høyskolen Kristiania

3.1 Bags of words

- The simplest vector space representational model for unstructured text?
- The Bag of Words model represents each text document as a numeric vector - each dimension is a specific word from the corpus and the value could be its frequency in the document, occurrence (denoted by 1 or 0), or even weighted values.

Høyskolen Kristiania

3.2. TF-IDF

- Stands for term frequency times inverse document frequency
  - Term frequencies are the counts of each word in a document
  - Inverse document frequency means that

TF-IDF

```
  (0, 5)        1
  (0, 9)        1
  (0, 10)       1
  (0, 7)        1
  (1, 6)        1
  (1, 2)        1
  (1, 1)        1
  (1, 12)       1
  (1, 3)        1
  (1, 8)        1
  (1, 0)        1
  (1, 4)        1
  (2, 5)        1
  (2, 9)        1
  (2, 10)       1
  (2, 7)        1
  (2, 11)       1
[[0 0 0 0 0 1 0 1 0 1 1 0 0]
 [1 1 1 1 1 0 1 0 1 0 0 0 1]
 [0 0 0 0 0 1 0 1 0 1 1 1 0]]
```

corpus = ['sentence1: similar to sentence3','sentence2: do'
'not care what other sentences are saying','sentence3:
very similar to sentence1']

['are', 'care', 'donot', 'other', 'saying', 'sentence1', 'sentence2',
'sentence3', 'sentences', 'similar', 'to', 'very', 'what']

Høyskolen Kristiania

3.2. TF-IDF

$$tf(w, D) = f_{w_D}$$

where f_{wD} denoted frequency for word **w** in document **D**, which becomes the term frequency (tf).

- TF: term frequency
- IDF: inverse document frequency

Høyskolen Kristiania

More thoughts

- By looking at an even more useful vector representation that counts the number of occurrences, or frequency, of each word in the given text, you assume that the more times a word occurs, the more meaning it must contribute to that document.
- Or if you have classified some words as expressing positive emotions—words like “good,” “best,” “joy,” and “fantastic”—the more a document that contains those words is likely to have positive “sentiment.”

Høyskolen Kristiania

Even more thoughts

- Let's say you find the word "dog" 3 times in document A and 100 times in document B. Clearly "dog" is way more important to document B while using TF.
- Wait!

$$TF(\text{“dog,”}\ document_A) = 3/30 = .1$$
$$TF(\text{“dog,”}\ document_B) = 100/580000 = .00017$$

Høyskolen Kristiania

- Instead of raw word counts to describe your documents in a corpus, use normalized term frequencies.

  For example, calculate each word and get the relative importance to the document of that term.

Høyskolen Kristiania

TF-IDF

$$\text{idf}(t, D) = \log \frac{\text{number of documents}}{\text{number of documents containing } t}$$

idf (w,D) represents the idf for the term/word w in document D, N represents the total number of documents in our corpus, and df(t) represents the number of documents in which the term w is present.

Høyskolen Kristiania

```
[[0.         0.         0.         0.         0.         0.5
  0.         0.5        0.         0.5        0.5        0.
  0.        ]
 [0.35355339 0.35355339 0.35355339 0.35355339 0.35355339 0.
  0.35355339 0.         0.35355339 0.         0.         0.
  0.35355339]
 [0.         0.         0.         0.         0.         0.41779577
  0.         0.41779577 0.         0.41779577 0.41779577 0.54935123
  0.        ]]
```

Høyskolen Kristiania

- Suppose you have a corpus of 1 million documents, someone searches for the word “cat,” and in your 1 million documents, you have exactly 1 document that contains the word “cat.” The raw IDF of this is

  1,000,000 / 1 = 1,000,000

- Suppose you have 10 documents with the word “dog” in them, the IDF for “dog” is:

  1,000,000 / 10 = 100,000

Høyskolen Kristiania

Similarly you can also implement IDF

- Be noted:

$$\text{idf}(t, D) = \log \frac{\text{number of documents}}{\text{number of documents containing } t}$$

$$\text{tf}(t, d) = \frac{\text{count}(t)}{\text{count}(d)}$$

$$\text{idf}(t, D) = \log \frac{\text{number of documents}}{\text{number of documents containing } t}$$

$$\text{tfidf}(t, d, D) = \text{tf}(t, d) * \text{idf}(t, D)$$

Kristiania

- TF-IDF for a document: K-dimensional vector representation of each document in the corpus.
- Calculate the similarity: two vectors are considered similar if their cosine similarity is high, so minimize:

$$\cos \Theta = \frac{A \cdot B}{|A|\ |B|}$$

Two vectors, in a given vector space, can be said to be similar if they have a similar angle. If you imagine each vector starting at the origin and reaching out its prescribed distance and direction, the ones that reach out at the same angle are similar, even if they don't reach out to the same distance.

Høyskolen Kristiania

- The more times a word appears in the document, the TF (and hence the TF-IDF) will go up
- The number of documents that contain that word goes up, the IDF (and hence the TF-IDF) for that word will go down
- In some classes, all the calculations will be done in log space so that multiplications become additions and division becomes subtraction

Høyskolen Kristiania

Similarity?

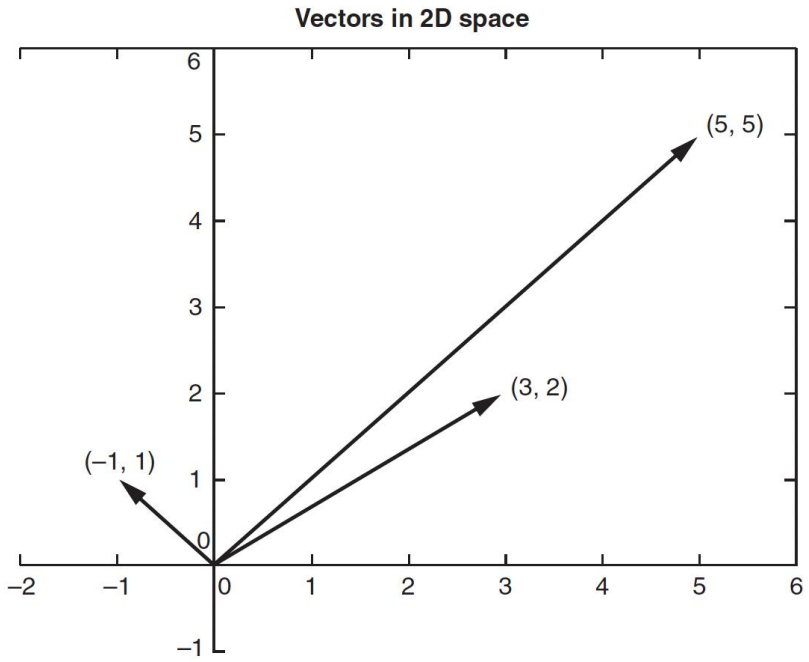
- Document similarity is the process of using a distance or similarity based metric that can identify how similar a text document is to any other document(s) based on features extracted from the documents, like Bag of Words (BoW) or TF-IDF.

Høyskolen Kristiania

- Pairwise document similarity in a corpus involves computing document similarity for each pair of documents in a corpus.
- Thus, if you have C documents in a corpus, you would end up with a C x C matrix, such that each row and column represents the similarity score for a pair of documents.

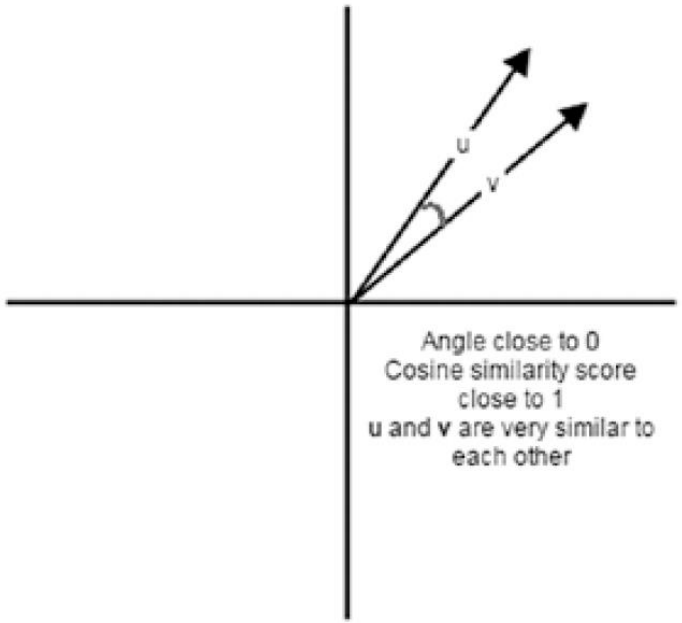
	0	1	2
0	1.000000	0.0	0.071144
1	0.000000	1.0	0.000000
2	0.071144	0.0	1.000000

Høyskolen Kristiania

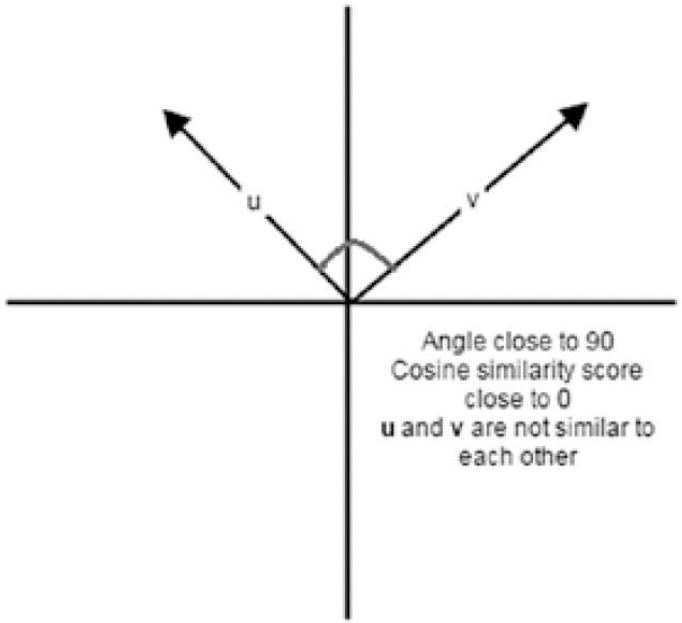


- Cosine similarity gives us a metric representing the cosine of the angle between the feature vector representations of two text documents.
- The smaller the angle between the documents, the closer and more similar they are.

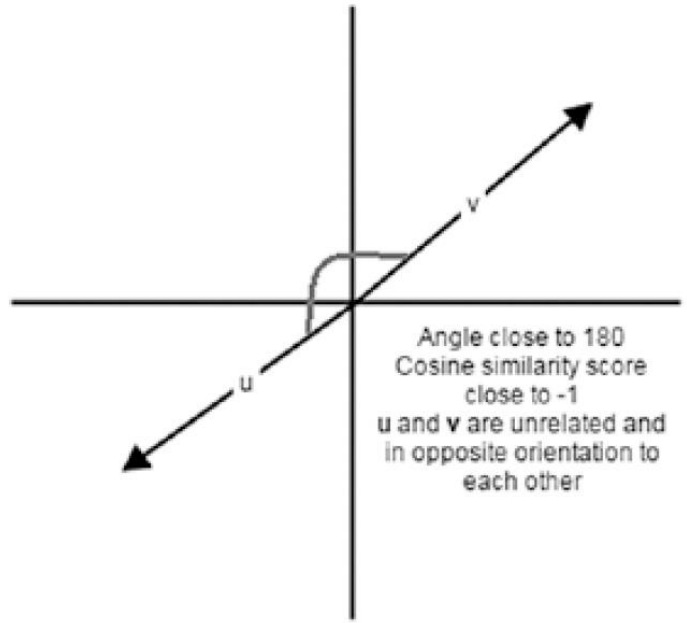
Høyskolen Kristiania



Angle close to 0
Cosine similarity score close to 1
u and **v** are very similar to each other



Angle close to 90
Cosine similarity score close to 0
u and **v** are not similar to each other



Angle close to 180
Cosine similarity score close to -1
u and **v** are unrelated and in opposite orientation to each other

Høyskolen Kristiania